E-Commerce ELT Data Pipeline Documentation

Project Title: End-to-End E-Commerce Analytics Pipeline using Snowflake & dbt

Prepared By: Shereen Mohamed Yasen

Executive Summary

This project demonstrates the design and implementation of an end-to-end ELT data pipeline. Raw e-commerce data (`customers`, `products`, and `orders`) was ingested into **Snowflake**, then cleaned, transformed, tested, and organized into analytics-ready data marts using **dbt (Data Build Tool)**.

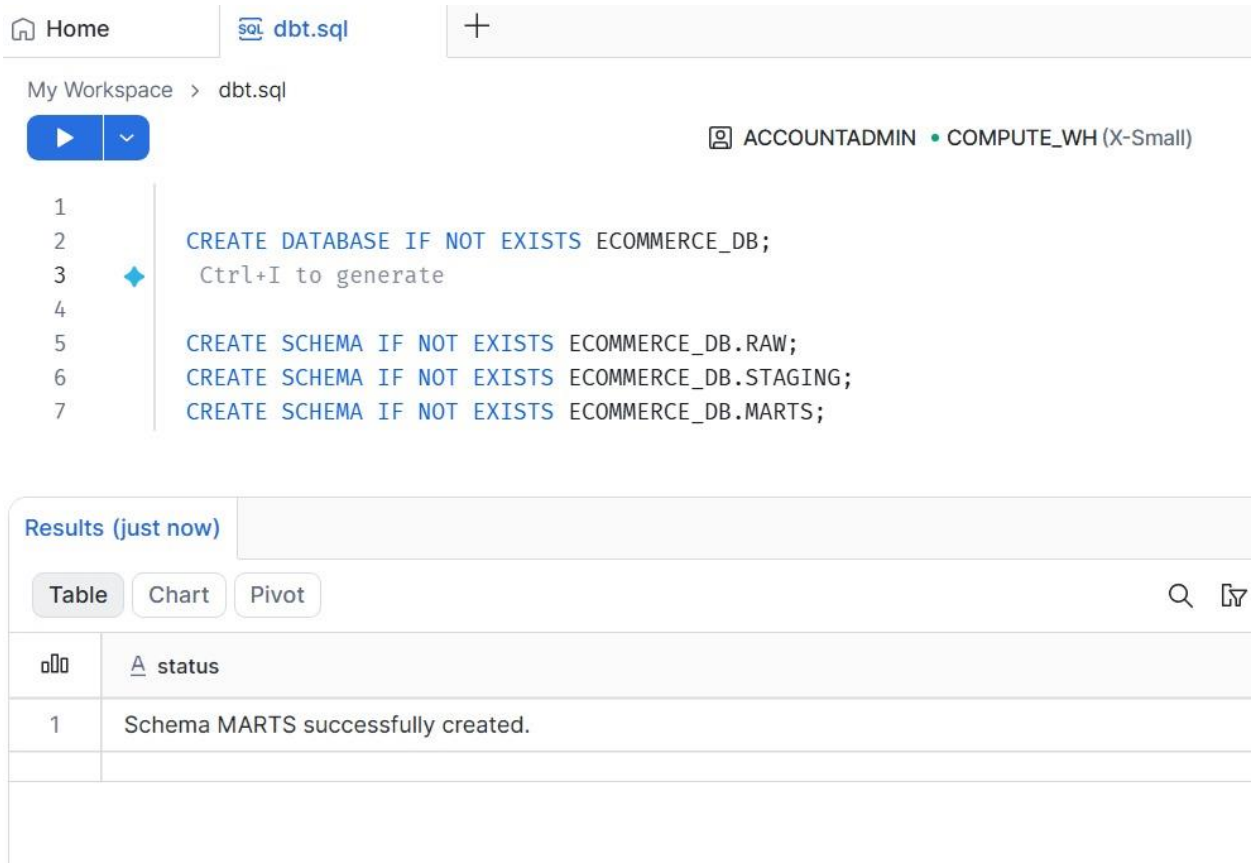
Task 1 — Snowflake Database Setup & Data Ingestion

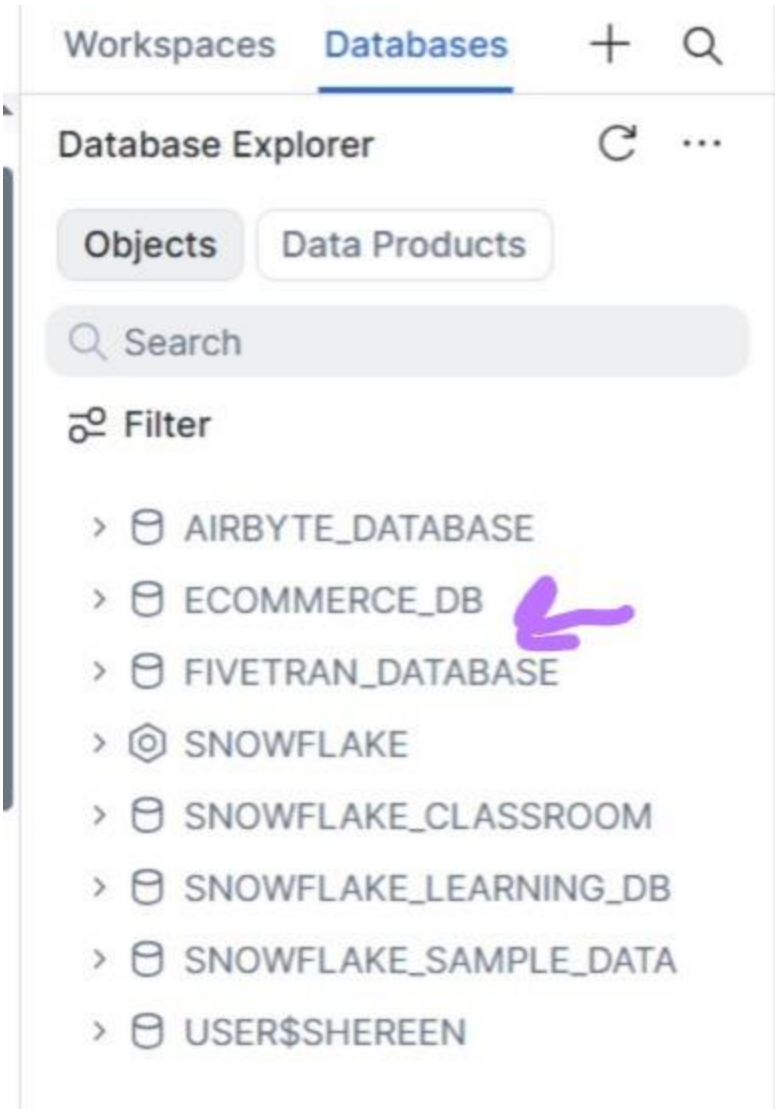
Objective

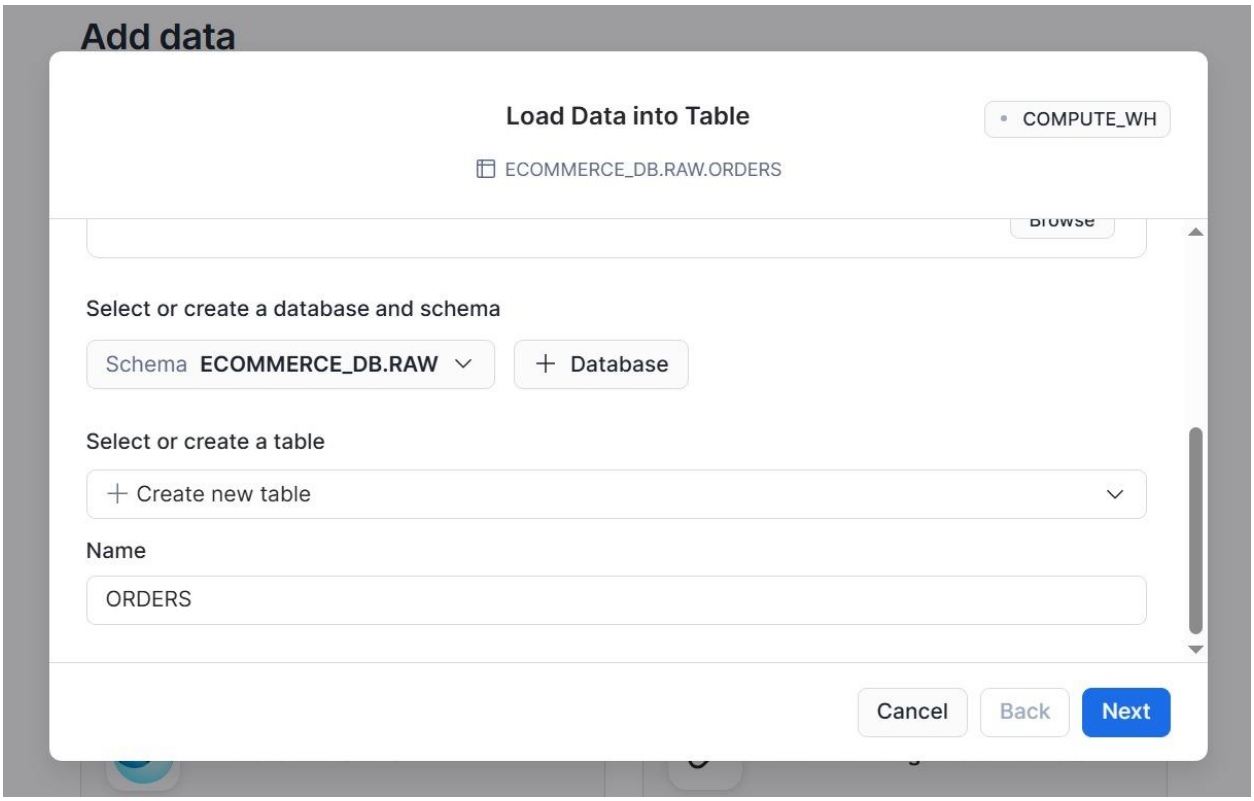
Establish the core data platform architecture and ingest source CSV files into the database.

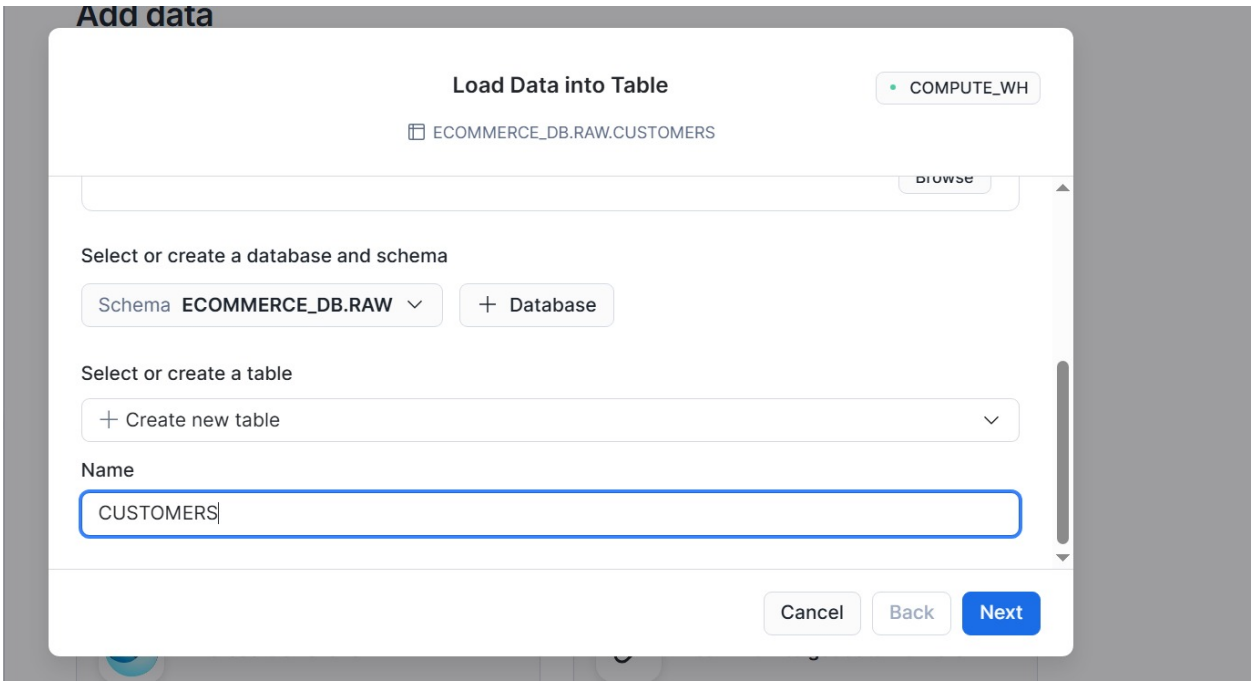
Actions Taken

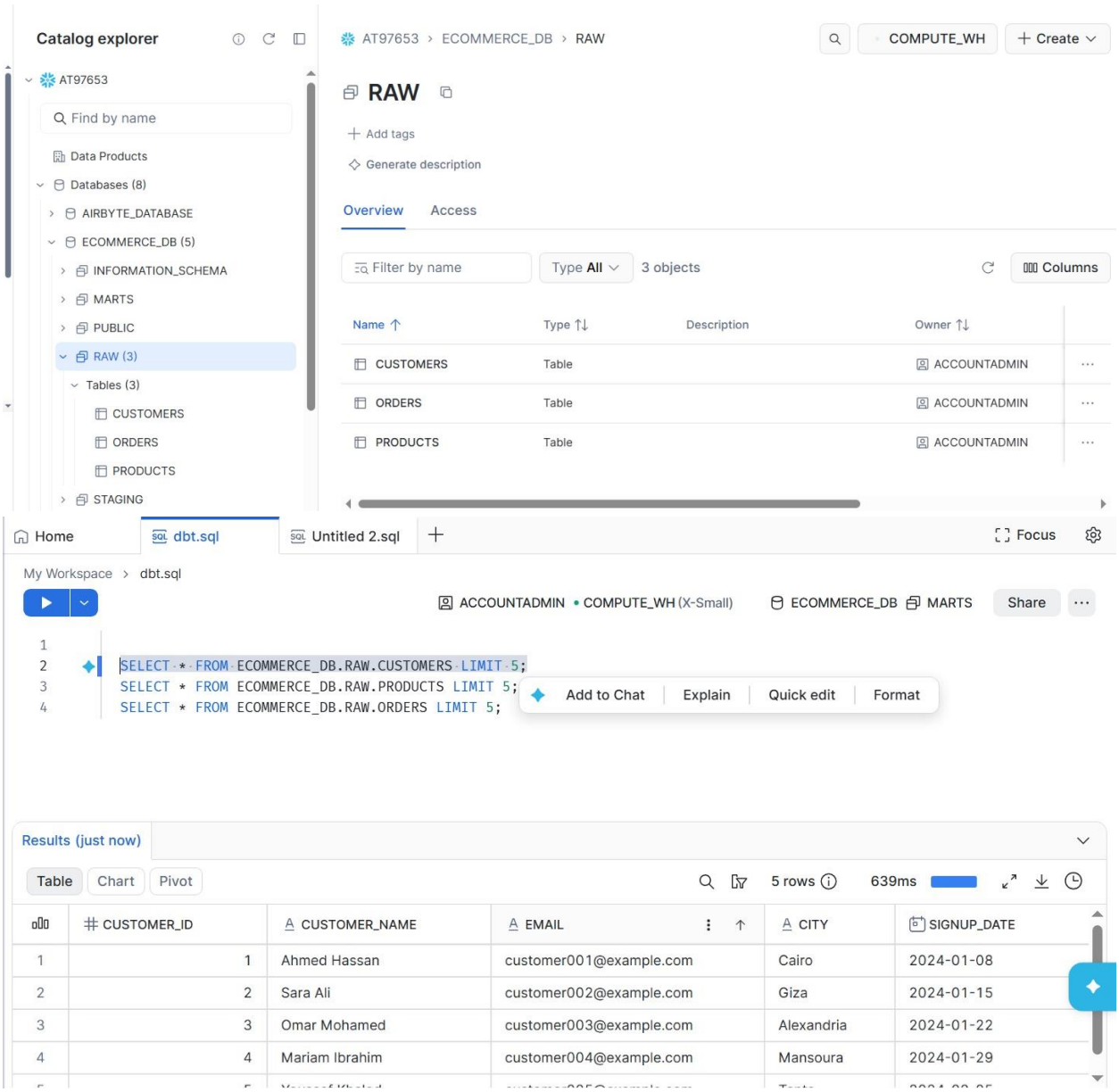
1. Created the target database `ECOMMERCE_DB`.
2. Created a 3-tier schema structure (`RAW`, `STAGING`, and `MARTS`) to support Medallion Data Architecture.
3. Ingested the three raw CSV files (`customers.csv`, `products.csv`, and `orders.csv`) into the `RAW` schema tables (`RAW.CUSTOMERS`, `RAW.PRODUCTS`, `RAW.ORDERS`).

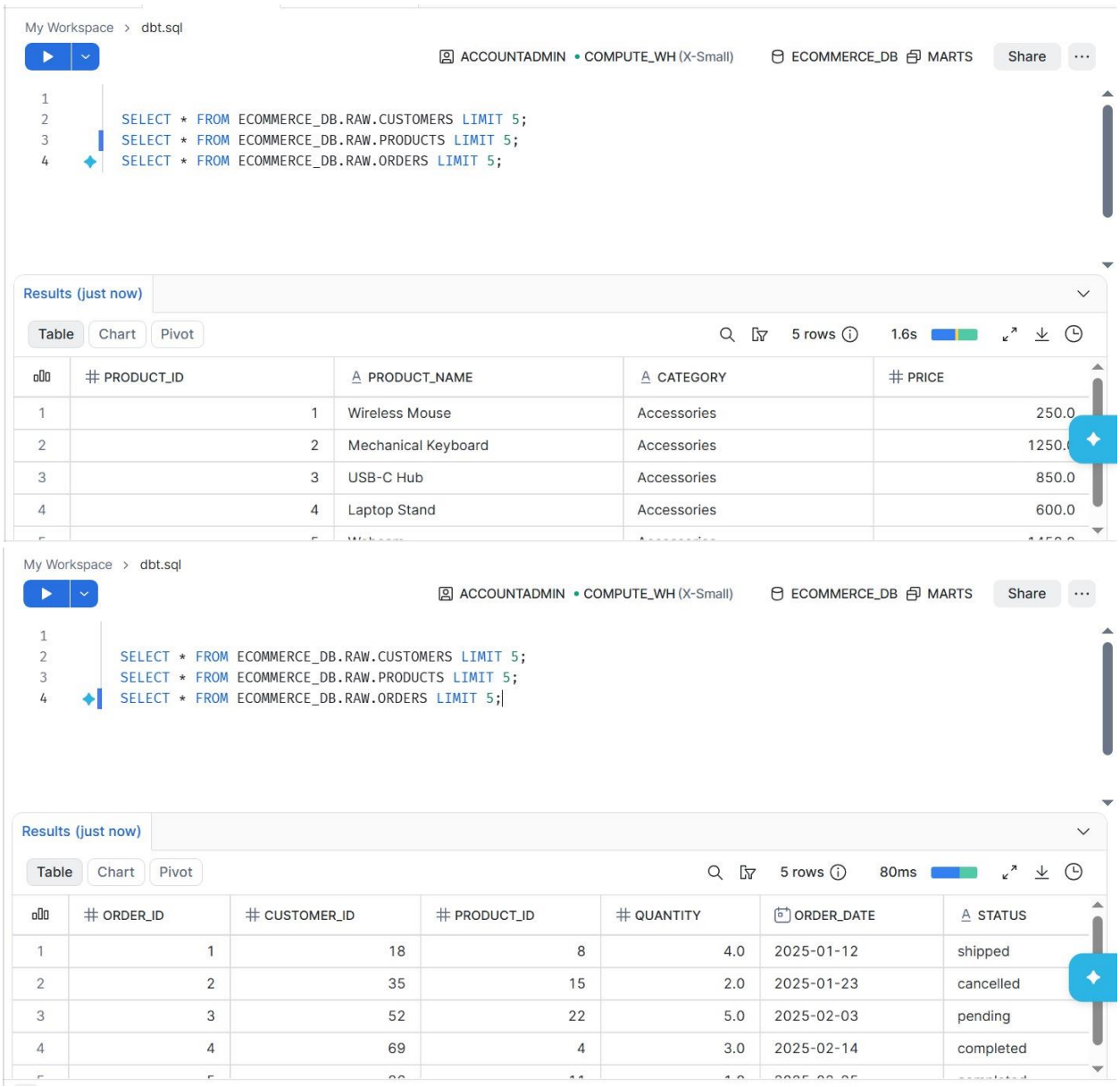

Home
dbt.sql
My Workspace > dbt.sql
ACCOUNTADMIN • COMPUTE_WH (X-Small)
1
2 CREATE DATABASE IF NOT EXISTS ECOMMERCE_DB;
3 Ctrl+I to generate
4
5 CREATE SCHEMA IF NOT EXISTS ECOMMERCE_DB.RAW;
6 CREATE SCHEMA IF NOT EXISTS ECOMMERCE_DB.STAGING;
7 CREATE SCHEMA IF NOT EXISTS ECOMMERCE_DB.MARTS;
Results (just now)
Table
Chart
Pivot
status
1
Schema MARTS successfully created.


Workspaces
Databases
Database Explorer
Objects
Data Products
Search
Filter
AIRBYTE_DATABASE
ECOMMERCE_DB
FIVETRAN_DATABASE
SNOWFLAKE
SNOWFLAKE_CLASSROOM
SNOWFLAKE_LEARNING_DB
SNOWFLAKE_SAMPLE_DATA
USER$SHEREEN


Add data
Load Data into Table
COMPUTE_WH
ECOMMERCE_DB.RAW.ORDERS
Browse
Select or create a database and schema
Schema ECOMMERCE_DB.RAW
Database
Select or create a table
Create new table
Name
ORDERS
Cancel
Back
Next


Add data
Load Data into Table
COMPUTE_WH
ECOMMERCE_DB.RAW.CUSTOMERS
Browse
Select or create a database and schema
Schema ECOMMERCE_DB.RAW
Database
Select or create a table
Create new table
Name
CUSTOMERS
Cancel
Back
Next


Catalog explorer
AT97653
Find by name
Data Products
Databases (8)
AIRBYTE_DATABASE
ECOMMERCE_DB (5)
INFORMATION_SCHEMA
MARTS
PUBLIC
RAW (3)
Tables (3)
CUSTOMERS
ORDERS
PRODUCTS
STAGING
AT97653 › ECOMMERCE_DB › RAW
COMPUTE_WH
Create
RAW
Add tags
Generate description
Overview
Access
Filter by name
Type All
3 objects
Columns
Name
Type
Description
Owner
CUSTOMERS
Table
ACCOUNTADMIN
ORDERS
Table
ACCOUNTADMIN
PRODUCTS
Table
ACCOUNTADMIN
Home
dbt.sql
Untitled 2.sql
Focus
My Workspace › dbt.sql
ACCOUNTADMIN • COMPUTE_WH (X-Small)
ECOMMERCE_DB
MARTS
Share
1
2
3
4
SELECT * FROM ECOMMERCE_DB.RAW.CUSTOMERS LIMIT 5;
SELECT * FROM ECOMMERCE_DB.RAW.PRODUCTS LIMIT 5;
SELECT * FROM ECOMMERCE_DB.RAW.ORDERS LIMIT 5;
Add to Chat
Explain
Quick edit
Format
Results (just now)
Table
Chart
Pivot
5 rows
639ms
CUSTOMER_ID
CUSTOMER_NAME
EMAIL
CITY
SIGNUP_DATE
1 1 Ahmed Hassan customer001@example.com Cairo 2024-01-08
2 2 Sara Ali customer002@example.com Giza 2024-01-15
3 3 Omar Mohamed customer003@example.com Alexandria 2024-01-22
4 4 Mariam Ibrahim customer004@example.com Mansoura 2024-01-29



Task 2 — Building dbt Staging Models

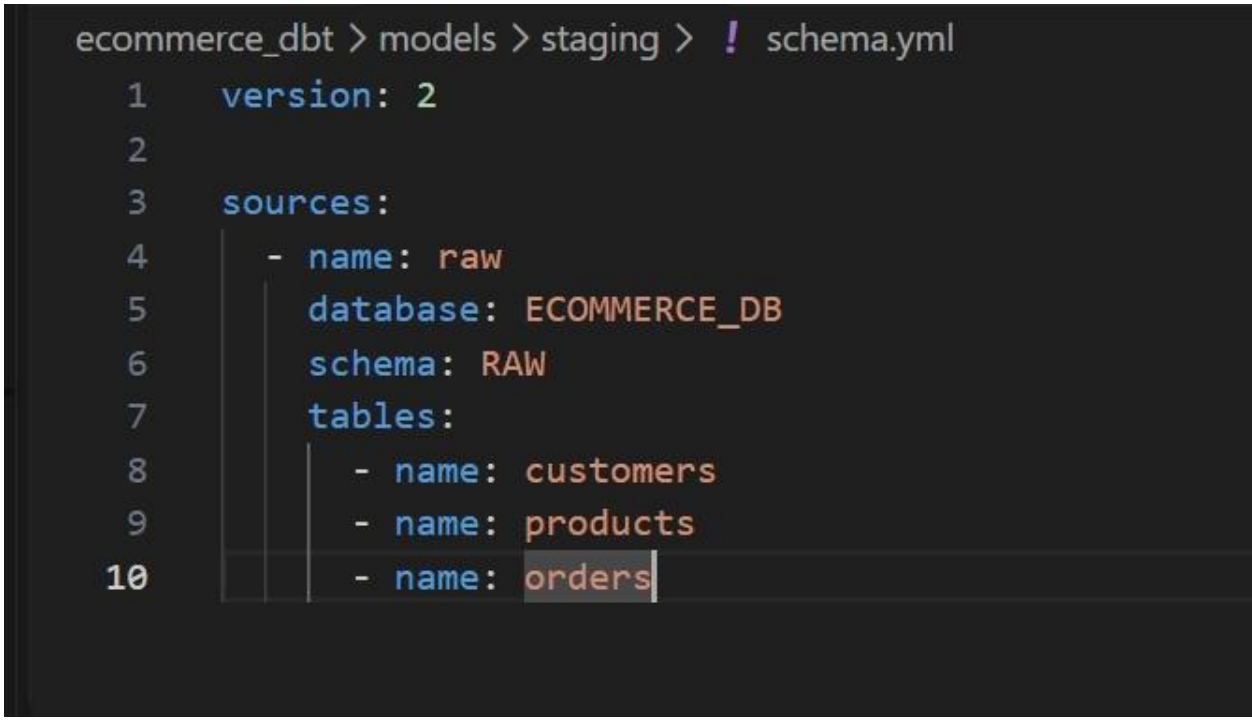
Objective

Standardize column naming, perform data type casting, and handle invalid or duplicate records.

Actions Taken

1. Configured raw tables as dbt sources inside `models/staging/schema.yml`.

2. Created three staging models (`stg_customers`, `stg_products`, `stg_orders`).
3. Handled raw data issues:
    a. **Column Standardization:** Mapped source columns.
    b. **Data Cleaning & Standardization:** Trimmed whitespaces, lowercased string columns, and standardized status values into `'completed'`, `'pending'`, and `'cancelled'`.
    c. **Deduplication:** Applied `ROW_NUMBER()` window function partitioned by `order_id` to deduplicate order entries.
    d. **Referential Integrity Filtering:** Filtered out invalid orders referencing non-existent `customer_id` or `product_id` keys.



ecommerce_dbt > models > staging > stg_customers.sql

```sql
select
    customer_id,
    trim(name) as customer_name,
    lower(trim(email)) as email,
    city,
    signup_date::date as signup_date
from {{ source('raw', 'customers') }}
```

ecommerce_dbt > models > staging > stg_products.sql

```sql
select
    product_id,
    trim(product_name) as product_name,
    category,
    price::decimal(10,2) as price
from {{ source('raw', 'products') }}
```

ecommerce_dbt > models > staging > stg_orders.sql

```sql
select
    customer_id,
    product_id,
    quantity::int as quantity,
    order_date::date as order_date,
    lower(trim(order_status)) as order_status
from {{ source('raw', 'orders') }}
```

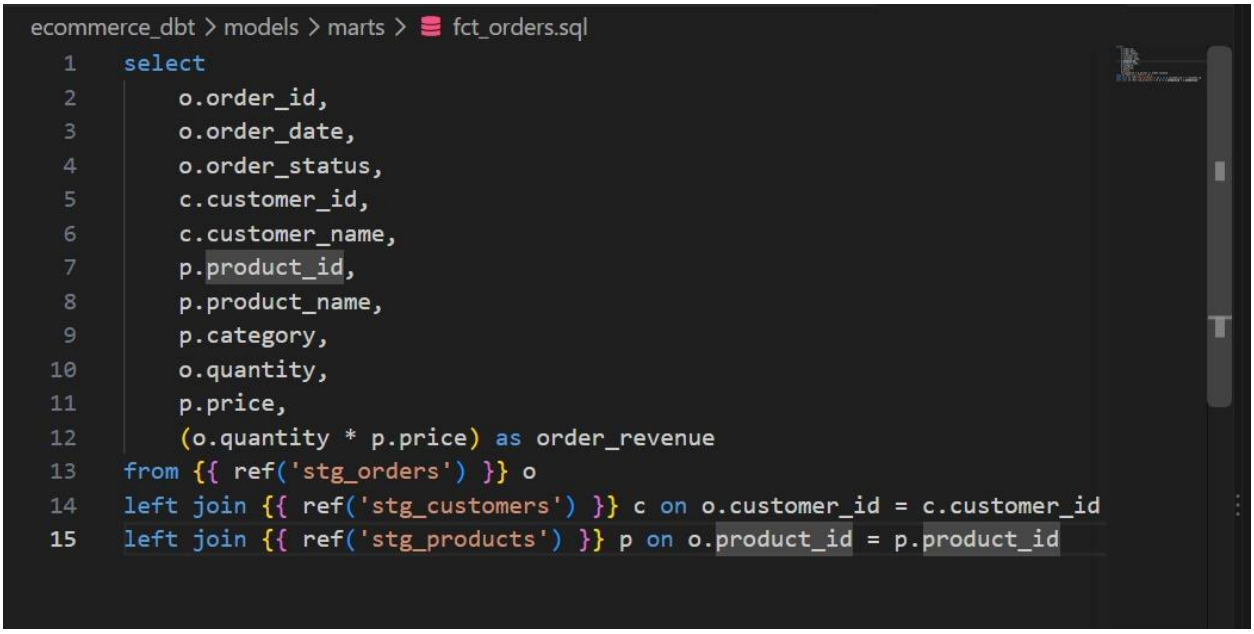
Task 3 & Task 4 — Transformations & Analytics Marts

Objective

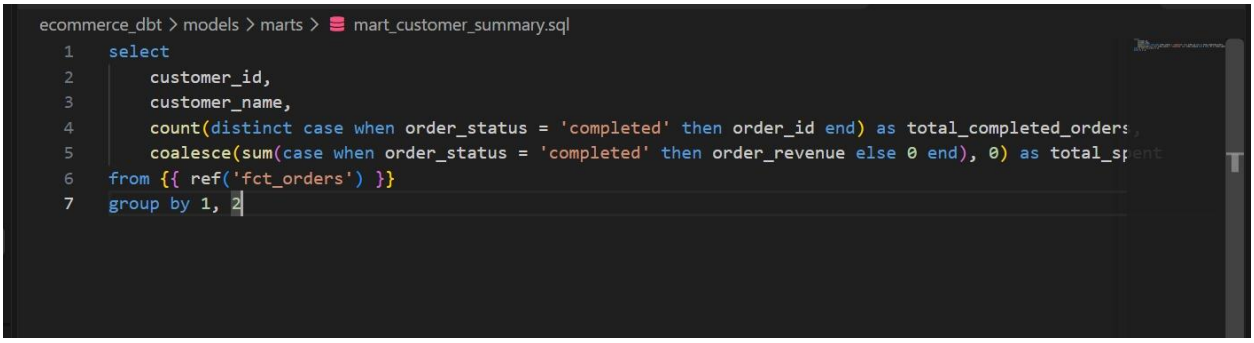
Denormalize core data into an order-fact table and aggregate business metrics into dimensional marts.

Implemented Models

1. **`fct_orders` (Dimensional Fact Model):** Joins `stg_orders` with customer and product details using `dbt ref()`. Computes total line-item revenue



2. **`mart_customer_summary` (Customer Analytical Mart):** Aggregates lifetime completed orders count and total spending per customer.



3. **`mart_category_sales` (Product Category Mart):** Aggregates order count and total revenue generated per product category.

ecommerce_dbt > models > marts > mart_category_sales.sql

```sql
select
    category,
    count(distinct case when order_status = 'completed' then order_id end) as completed_orders_count,
    coalesce(sum(case when order_status = 'completed' then order_revenue else 0 end), 0) as total_category_revenue
from {{ ref('fct_orders') }}
group by 1
```

ecommerce_dbt > models > marts > ! schema.yml

```yaml
version: 2

models:
  - name: stg_customers
    columns:
      - name: customer_id
        tests:
          - unique
          - not_null

  - name: stg_products
    columns:
      - name: product_id
        tests:
          - unique
          - not_null

  - name: stg_orders
    columns:
      - name: order_id
        tests:
          - unique
          - not_null
      - name: customer_id
        tests:
          - not_null
          - relationships:
              to: ref('stg_customers')
              field: customer_id
```

Treatment of Order Statuses

- All orders (completed, pending, cancelled) are preserved in `fct_orders` for full order lifecycle visibility.

- Non-completed orders (`pending`, `cancelled`) are explicitly excluded from financial revenue metrics using conditional aggregations (`CASE WHEN order_status = 'completed' THEN order_revenue ELSE 0 END`) to avoid revenue inflation.

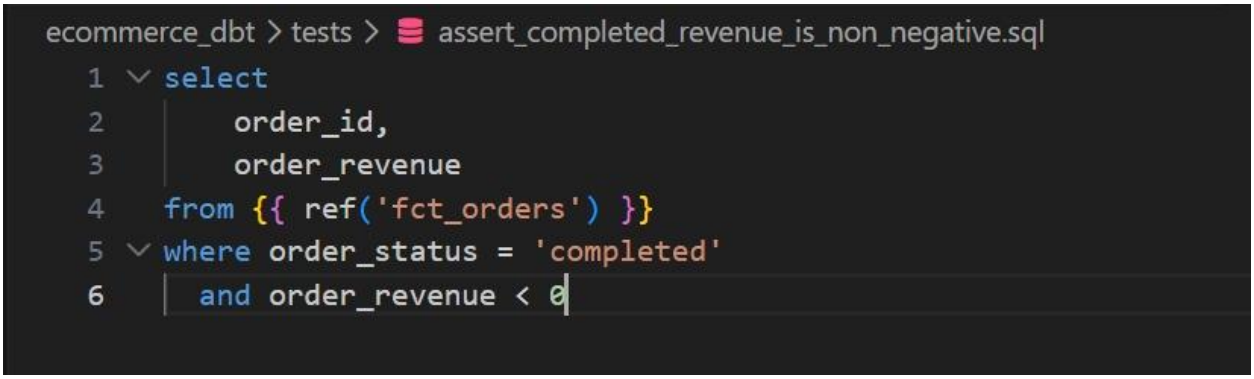
Task 5 — Data Quality Testing

Objective

Validate primary keys, foreign key relationships, valid status values, and custom business logic.

Applied Generic Tests

- **Uniqueness & Not-Null:** Applied to `customer_id`, `product_id`, and `order_id` across staging and fact tables.
- **Referential Integrity:** Tested via `relationships` tests ensuring `stg_orders.customer_id` and `stg_orders.product_id` exist in their dimension models.
- **Accepted Values:** Verified that `order_status` only contains `['completed', 'pending', 'cancelled']`.



Task 6 — Pipeline Execution & Rerun Strategy

Pipeline Execution Summary

Executed full transformation and testing cycle:

1. `dbt run`: Compiled SQL models and materialized views/tables inside Snowflake.

```
Problems  Output  Debug Console  Terminal  Ports        powershell - ecommerce_dbt

PS C:\snowflake-dbt\ecommerce_dbt> dbt run
20:24:41  2 of 6 OK created sql view model STAGING.stg_orders ........................... [SUCCESS 1 in 1.69s
]
20:24:41  4 of 6 START sql view model STAGING.fct_orders ................................. [RUN]
20:24:41  4 of 6 OK created sql view model STAGING.fct_orders ........................... [SUCCESS 1 in 0.83s
]
20:24:41  5 of 6 START sql view model STAGING.mart_category_sales ........................ [RUN]
20:24:41  6 of 6 START sql view model STAGING.mart_customer_summary ...................... [RUN]
20:24:42  6 of 6 OK created sql view model STAGING.mart_customer_summary ................. [SUCCESS 1 in 0.43s
]
20:24:42  5 of 6 OK created sql view model STAGING.mart_category_sales ................... [SUCCESS 1 in 0.48s
]
20:24:43
20:24:43  Finished running 6 view models in 0 hours 0 minutes and 6.33 seconds (6.33s).
20:24:43
20:24:43  Completed successfully
20:24:43
20:24:43  Done. PASS=6 WARN=0 ERROR=0 SKIP=0 NO-OP=0 REUSED=0 TOTAL=6
PS C:\snowflake-dbt\ecommerce_dbt>
```

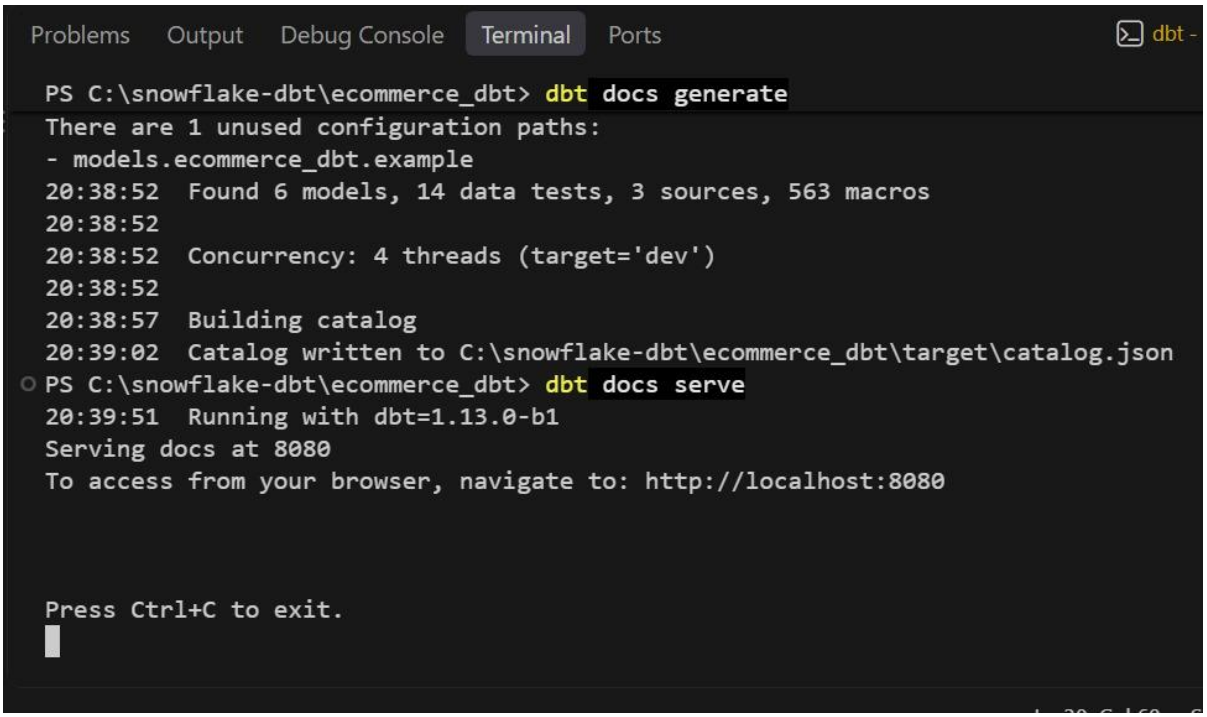
2. `dbt test`: Verified all generic and custom data quality constraints.

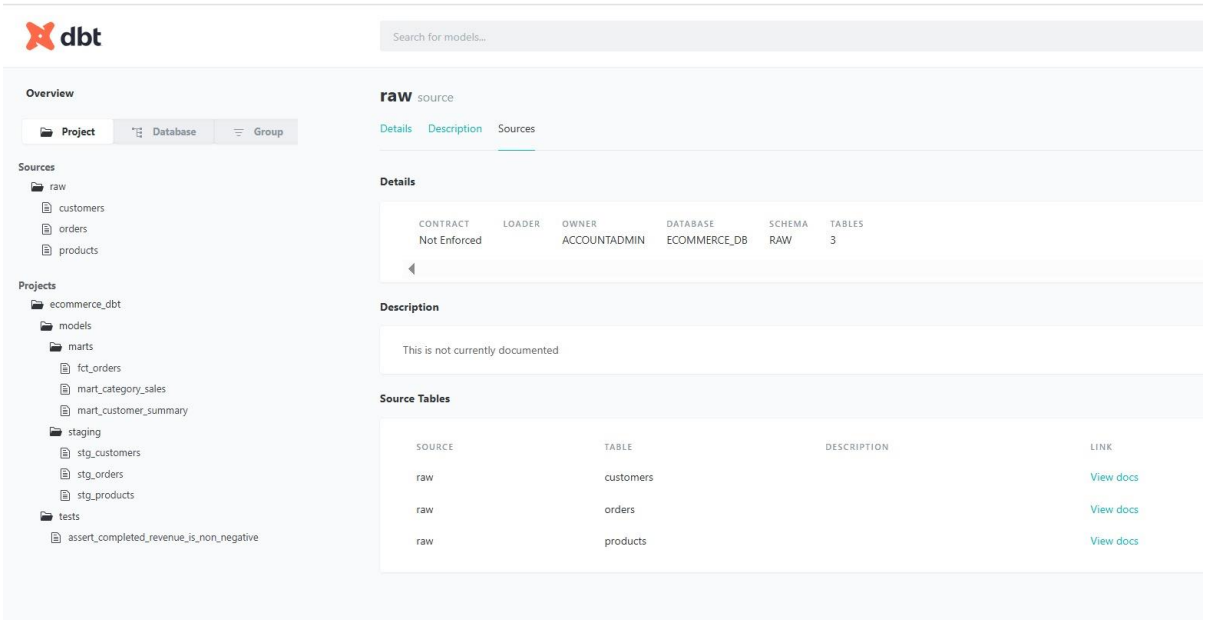
```
Problems  Output  Debug Console  Terminal  Ports        powershell - ecommerce_dbt

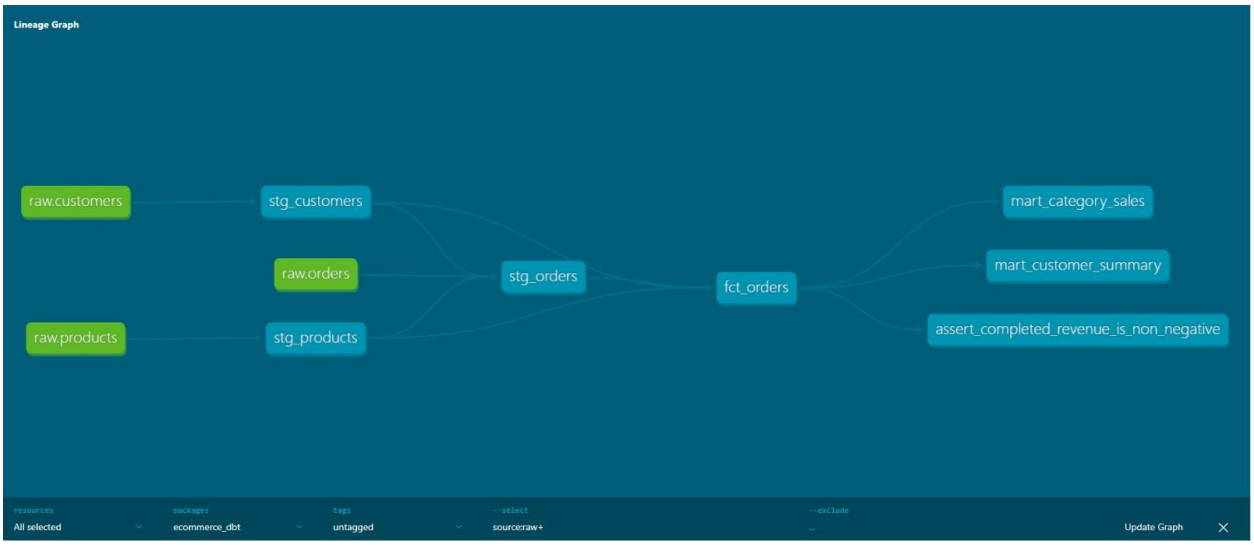
PS C:\snowflake-dbt\ecommerce_dbt> dbt test
20:36:12  11 of 14 START test unique_fct_orders_order_id ................................ [RUN]
20:36:12  12 of 14 START test unique_stg_customers_customer_id ........................... [RUN]
20:36:12  12 of 14 PASS unique_stg_customers_customer_id ................................. [PASS in 0.36s]
20:36:12  13 of 14 START test unique_stg_orders_order_id ................................. [RUN]
20:36:13  9 of 14 PASS relationships_stg_orders_customer_id__customer_id__ref_stg_customers_  [PASS in 0.89s]
20:36:13  10 of 14 PASS relationships_stg_orders_product_id__product_id__ref_stg_products_  [PASS in 0.65s]
20:36:13  14 of 14 START test unique_stg_products_product_id .............................. [RUN]
20:36:13  11 of 14 PASS unique_fct_orders_order_id ....................................... [PASS in 0.79s]
20:36:13  13 of 14 PASS unique_stg_orders_order_id ....................................... [PASS in 0.35s]
20:36:13  14 of 14 PASS unique_stg_products_product_id ................................... [PASS in 0.35s]
20:36:14
20:36:14  Finished running 14 data tests in 0 hours 0 minutes and 6.19 seconds (6.19s).
20:36:14
20:36:14  Completed successfully
20:36:14
20:36:14  Done. PASS=14 WARN=0 ERROR=0 SKIP=0 NO-OP=0 REUSED=0 TOTAL=14
PS C:\snowflake-dbt\ecommerce_dbt>

Ln 38, Col 60   Spaces: 4   UTF-8   CRLF   { } MS SQL
```

3. `dbt docs generate` & `dbt docs serve`: Generated interactive documentation and dependencies catalog.


Problems
Output
Debug Console
Terminal
Ports
dbt -
PS C:\snowflake-dbt\ecommerce_dbt> dbt docs generate
There are 1 unused configuration paths:
- models.ecommerce_dbt.example
20:38:52 Found 6 models, 14 data tests, 3 sources, 563 macros
20:38:52
20:38:52 Concurrency: 4 threads (target='dev')
20:38:52
20:38:57 Building catalog
20:39:02 Catalog written to C:\snowflake-dbt\ecommerce_dbt\target\catalog.json
PS C:\snowflake-dbt\ecommerce_dbt> dbt docs serve
20:39:51 Running with dbt=1.13.0-b1
Serving docs at 8080
To access from your browser, navigate to: http://localhost:8080
Press Ctrl+C to exit.


dbt
Search for models...
Overview
Project
Database
Group
Sources
raw
customers
orders
products
Projects
ecommerce_dbt
models
marts
fct_orders
mart_category_sales
mart_customer_summary
staging
stg_customers
stg_orders
stg_products
tests
assert_completed_revenue_is_non_negative
raw source
Details
Description
Sources
Details
CONTRACT
Not Enforced
LOADER
OWNER
ACCOUNTADMIN
DATABASE
ECOMMERCE_DB
SCHEMA
RAW
TABLES
3
Description
This is not currently documented
Source Tables
SOURCE
TABLE
DESCRIPTION
LINK
raw
customers
View docs
raw
orders
View docs
raw
products
View docs



Pipeline Rerun Strategy (Handling Incremental Data Arrival)

When new order records arrive in Snowflake `RAW.ORDERS`:

- **Current Setup (Full Refresh):** Executing `dbt run` automatically picks up newly appended raw rows, applies cleaning rules, deduplicates, and re-materializes all downstream marts.
- **Production Scaling Option (Incremental Materialization):** For large-scale data, `stg_orders` and `fct_orders` can be configured with `materialized='incremental'` using `order_date` watermarks to process only newly added records without re-processing historical data.